Software Architecture and Design Specification

IEEE-style architecture and design specification

Project: Pharmacy Expiry & Re-order Dispatch Engine

Version: 1.0 | Status: Student Submission Draft | Date: 12 October 2026

Prepared for: Software Engineering Mini-Project Deliverables – Part 1

Revision History

Version	Date	Author	Change
1.0	12-Oct-2026	Student	Initial submission draft

1. Introduction

1.1 Purpose

This document describes the architecture and detailed design for the Pharmacy Expiry & Re-order Dispatch Engine.

1.2 Scope

The system manages medicine batches, inventory views, FEFO lists, reorder thresholds, and purchase orders.

1.3 Audience

Developers, QA engineers, project evaluators, maintainers, and pharmacy stakeholders.

1.4 Definitions

FEFO: First-Expiry-First-Out. ADR: Architecture Decision Record. RBAC: Role-Based Access Control.

2. Document Overview

This specification records architectural goals, stakeholders, component responsibilities, pattern selection, traceability, security controls, sequence flows, API contracts, error handling, and monitoring.

3. Architecture

3.1 Goals and Constraints

Goals: correct expiry-aware stock selection, inventory integrity, maintainability, traceability, and responsive queries.
Constraints: student-project scope, accurate batch/expiry data, and no guaranteed live supplier integration.

3.2 Stakeholders and Concerns

Stakeholder	Concern
Pharmacy Clerk	Fast, clear batch entry and FEFO list.
Inventory Manager	Accurate stock alerts and controlled purchase orders.
Supplier	Clear order information and dispatch status.
Developer/Maintainer	Modularity, testability, and clear interfaces.
Evaluator/QA	Traceability, measurable requirements, and test evidence.

3.3 Component Diagram

Component responsibilities and interactions:

```
@startuml
skinparam componentStyle rectangle
actor "Pharmacy Clerk" as Clerk
actor "Inventory Manager" as Manager
actor "Inventory Supplier" as Supplier
component "User Interface" as UI
component "Authentication & RBAC" as Auth
component "Batch/Inventory Service" as Inv
component "FEFO Selector" as FEFO
component "Reorder Monitor" as Reorder
component "Purchase Order Service" as PO
database "Pharmacy Database" as DB
```

```
Clerk --> UI
Manager --> UI
UI --> Auth
UI --> Inv
UI --> FEFO
UI --> Reorder
UI --> PO
Inv --> DB
FEFO --> Inv
Reorder --> Inv
Reorder --> DB
PO --> DB
PO --> Supplier : order details/status
@enduml
```

Component	Responsibility
User Interface	Collects user input and displays inventory, FEFO results, alerts, and orders.
Authentication & RBAC	Authenticates users and enforces role permissions.
Batch/Inventory Service	Validates and stores batch details; provides stock summaries.
FEFO Selector	Filters expired/zero-stock batches and sorts eligible batches by expiry date.
Reorder Monitor	Compares available stock with configured threshold.
Purchase Order Service	Creates orders, tracks status, and records dispatch details.
Pharmacy Database	Persists medicines, batches, thresholds, suppliers, users/roles, and orders.

3.4 Chosen Architecture Pattern and Rationale

A layered architecture is selected: presentation, application/service, domain logic, and persistence. This separates UI concerns from business rules and database access, making unit testing and maintenance easier. A microservices architecture is not selected because the project scope does not justify distributed deployment and operational complexity.

3.5 Technology Stack and Data Stores

Logical stack: browser/desktop user interface; application service layer; relational database. The exact programming language, framework, and database should match the implementation repository and must be recorded by the team before final deployment. This specification does not assume an unverified stack.

3.6 Risks and Mitigations

Risk	Mitigation
Incorrect expiry data	Validate date formats and make batch details reviewable.
Race conditions in stock changes	Use database transactions and re-check quantity before committing changes.
Unauthorized changes	Enforce authentication and role checks on server-side actions.
Supplier integration unavailable	Record order status and provide a manual dispatch workflow.
Incorrect FEFO output	Unit-test expiry boundary, zero stock, and sorting rules.

3.7 Traceability to Requirements

Requirement	Design component(s)	Verification
FR-01	Batch/Inventory Service, Database	TC-01, TC-02
FR-02	Batch/Inventory Service, UI	TC-03
FR-03	FEFO Selector	TC-04, TC-05
FR-04	Reorder Monitor	TC-06
FR-05	Purchase Order Service	TC-07, TC-08
NFR-01	Inventory Service, FEFO Selector	TC-09
NFR-02 / SEC-02	Authentication & RBAC	TC-08, TC-10

3.8 Security Architecture

Use a STRIDE-inspired review: Spoofing—authenticate each user; Tampering—validate input and use transactions; Information Disclosure—restrict data and logs; Denial of Service—bound input size and rate where supported; Elevation of Privilege—enforce server-side RBAC. Credentials and secrets must not be logged. Audit changes using actor, timestamp, action, and record identifier.

4. Design

4.1 Design Overview

The UI calls application services. Services validate requests and invoke domain rules. Domain logic performs FEFO filtering/sorting and reorder comparisons. Repositories persist changes. The service layer is responsible for authorization and transaction boundaries.

4.2 UML Sequence Diagrams

4.2.1 Generate FEFO Dispensing List

```
@startuml
actor "Pharmacy Clerk" as Clerk
participant UI
participant "Auth/RBAC" as Auth
participant "FEFO Selector" as FEFO
participant "Inventory Service" as Inv
database DB
Clerk -> UI : select medicine
UI -> Auth : authorize request
Auth --> UI : permitted
UI -> FEFO : getEligibleBatches(medicineId)
FEFO -> Inv : get batches
Inv -> DB : query batches
DB --> Inv : batch records
Inv --> FEFO : batch records
FEFO -> FEFO : exclude expired/zero stock; sort by expiry
FEFO --> UI : ordered eligible batches
UI --> Clerk : display FEFO list
@enduml
```

4.2.2 Generate Purchase Order

```
@startuml
actor "Inventory Manager" as Manager
participant UI
participant "Auth/RBAC" as Auth
participant "Reorder Monitor" as Reorder
participant "Purchase Order Service" as PO
database DB
Manager -> UI : create order
UI -> Auth : authorize manager action
Auth --> UI : permitted
UI -> Reorder : check stock/threshold
Reorder -> DB : read stock and threshold
DB --> Reorder : current values
Reorder --> UI : low-stock result
UI -> PO : createOrder(medicine, quantity, supplier)
PO -> PO : validate required fields
PO -> DB : save order transactionally
DB --> PO : order ID
PO --> UI : order ID and status
```

UI --> Manager : show confirmation
@enduml

4.3 API Design

Interface	Method / operation	Request	Response / errors
Batch Management	POST /batches	medicineId, batchId, expiryDate, quantity	201 batch record; 400 invalid fields; 401/403 unauthorized; 409 duplicate batch.
FEFO List	GET /medicines/{medicineId}/fefo	medicineId path parameter	200 ordered eligible batches; 404 unknown medicine; 401/403 unauthorized.
Low Stock	GET /inventory/low-stock	Optional pagination/filter	200 list of medicines below threshold; 401/403 unauthorized.
Purchase Order	POST /purchase-orders	medicineId, quantity, supplierId	201 orderId/status; 400 invalid data; 401/403 unauthorized; 404 unknown medicine/supplier.

4.4 Error Handling, Logging and Monitoring

Return consistent client-safe error responses with a code, short message, and correlation/request ID. Do not expose stack traces or database details to end users. Log validation failures and state-changing actions without secrets. Monitor failed requests, FEFO request latency, low-stock alert count, order creation failures, and database availability.

4.5 UX Design

Use clear field labels, explicit expiry dates, visual but text-labelled expired/low-stock statuses, confirmation before order submission, keyboard-accessible controls, and actionable validation messages. Do not rely on color alone to communicate status.

4.6 Open Issues and Next Steps

Confirm implementation stack and endpoint naming against the actual code; decide whether supplier dispatch is manual or integrated; define audit-log retention; conduct usability review with representative users; implement and run all traceability-linked tests.

5. Appendices

5.1 Glossary

FEFO—First-Expiry-First-Out; RBAC—Role-Based Access Control; SRS—Software Requirements Specification; STP—Software Test Plan; RTM—Requirements Traceability Matrix.

5.2 References

- IEEE 42010 architecture description concepts (as referenced by the supplied template).
- OWASP secure design and input-validation guidance.
- NIST SP 800-160 systems security engineering (as referenced by the supplied template).

5.3 Tools

PlantUML or draw.io for UML diagrams; Swagger/OpenAPI if HTTP APIs are implemented; test runner and issue tracker for verification.